

Computation Graphs

DS 4440 · LECTURE 4

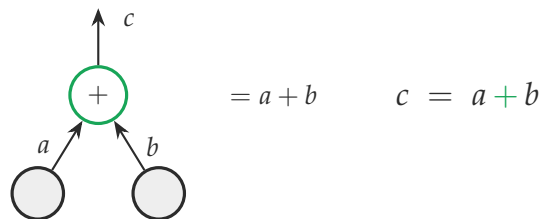
1. So far

- Supervised learning
- **SGD** for linear models
- **MLPs** and non-linear models

Today: **computation graphs** as the core abstraction for **neural networks**.

2. The Big Idea^(TM)

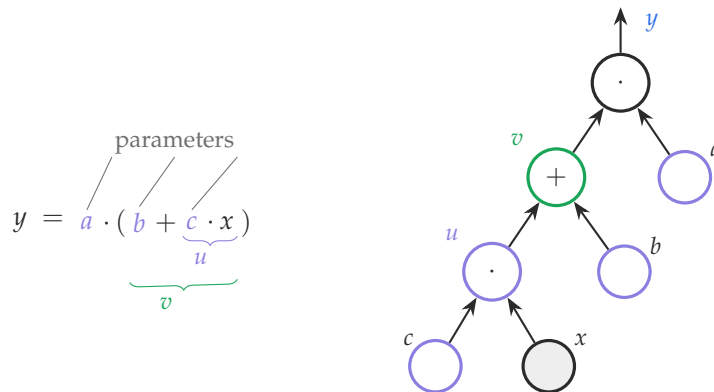
Models as **directed acyclic graphs** (DAGs).



Nodes are **operations** (possibly the identity), labeled as **variables**. **Tensors** — scalars in the examples above — “flow” along the edges.

We care about **forward** and **backward** passes through such graphs.

Consider

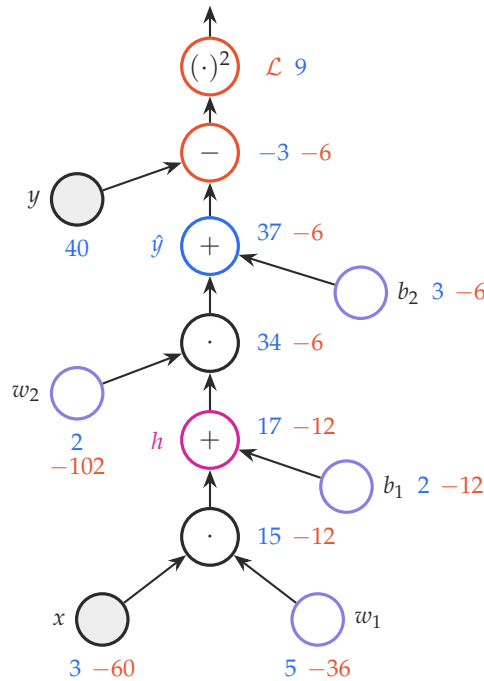


3. A graph for a simple regression model

Exercise! See the notebook.

$$\hat{y} = w_2 \cdot (w_1 \cdot x + b_1) + b_2, \quad \mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2.$$

Take $x = 3$, $w_1 = 5$, $b_1 = 2$, $w_2 = 2$, $b_2 = 3$, and $y = 40$. **Forward** values are in blue, and **backward** values — computed in §5 — in orange.



4. Differentiating by hand

Take the gradient with respect to w_2 , substituting the model in and applying the chain rule:

$$\begin{aligned}
 \frac{\partial}{\partial w_2} \mathcal{L} &= \frac{\partial}{\partial w_2} (\hat{y} - y)^2 \\
 &= \frac{\partial}{\partial w_2} \left(\underbrace{(w_2 \cdot [w_1 \cdot x + b_1] + b_2) - y}_u \right)^2 \\
 &= \frac{\partial}{\partial u} u^2 \cdot \frac{\partial}{\partial w_2} \left(w_2 \underbrace{[w_1 \cdot x + b_1]}_h + b_2 - y \right) \\
 &= 2(\hat{y} - y) \cdot h.
 \end{aligned}$$

With the numbers above, $2(37 - 40) \cdot 17 = -102$, which is the value attached to w_2 in the graph.

5. The local rule

Doing that substitution once per parameter does not scale — and it is not what the graph is for. The point of the graph is that **each node only needs to know three things**: its inputs, its output, and the derivative of its output with respect to its inputs. Nothing global. The three operations in our graph:

node	local derivative	so the gradient...
$c = a + b$	$\partial c / \partial a = \partial c / \partial b = 1$	passes through unchanged to both inputs
$c = a \cdot b$	$\partial c / \partial a = b, \partial c / \partial b = a$	goes to each input times the <i>other</i> input
$c = u^2$	$\partial c / \partial u = 2u$	is multiplied by $2u$

gradient at an input = (incoming gradient) $\times$ (local derivative)

The full backward sweep

Start at the top with $\partial \mathcal{L} / \partial \mathcal{L} = 1$ and walk down. Every line is one application of the rule above:

node	(incoming) $\times$ (local)	$\partial \mathcal{L} / \partial \cdot$
$\hat{y}$	$1 \cdot 2(\hat{y} - y) = 2(37 - 40)$	-6
b_2	$-6 \cdot 1$	-6
w_2	$-6 \cdot \mathbf{h} = -6 \cdot 17$	-102
$\mathbf{h}$	$-6 \cdot w_2 = -6 \cdot 2$	-12
b_1	$-12 \cdot 1$	-12
w_1	$-12 \cdot x = -12 \cdot 3$	-36
x	$-12 \cdot w_1 = -12 \cdot 5$	-60

One sweep gives every gradient. A few things fall out of this:

- **The forward pass has to be cached.** $\partial \mathcal{L} / \partial w_2$ needs $\mathbf{h} = 17$, a *forward* quantity. The graph holds on to its intermediate values until the backward pass consumes them — which is why training uses far more memory than inference.
- **Reverse is the cheap direction.** One backward pass produces gradients for all parameters at once. Differentiating forward would cost one pass *per* parameter; with one scalar loss and millions of parameters, reverse mode is the only viable choice.
- **Leaves differ.** We want $\partial \mathcal{L} / \partial w_1$ to update the parameter. We computed $\partial \mathcal{L} / \partial x = -60$ too, but nothing uses it — x is data, not something we optimize.
- **Acyclic matters.** It is what guarantees an evaluation order exists, both forward and in reverse.

When a node has two consumers

Our graph is a tree, so every value feeds exactly one node. In general it will not be, and then the gradients from each consumer **add**. For $f = a \cdot b + a$, the variable a feeds two nodes, and

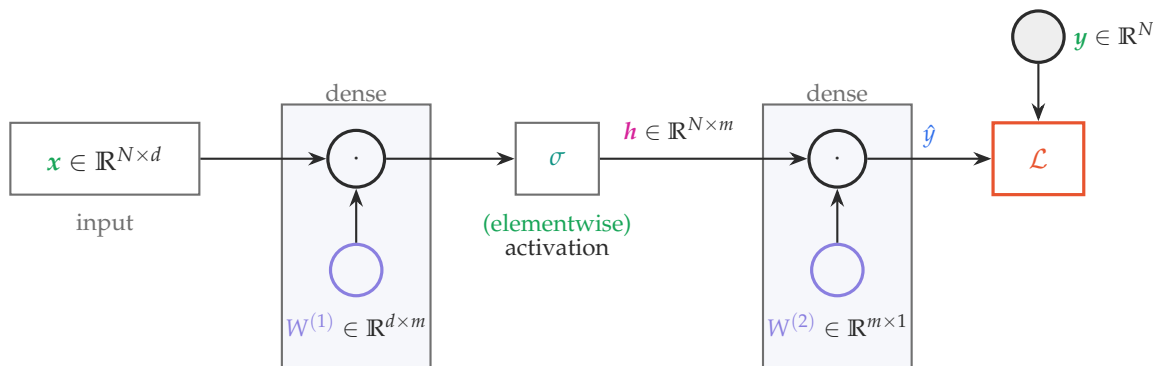
$$\frac{\partial f}{\partial a} = \underbrace{b}_{\text{via } a \cdot b} + \underbrace{1}_{\text{via } +a} = b + 1.$$

This is the same fact as gradient *accumulation* in code, and it is why we have to zero out gradients between steps.

6. From scalars to tensors

We have been considering scalars, but in general we should think of **tensors**:

$$\hat{y} = W^{(2)} \cdot \sigma(W^{(1)} \cdot x).$$



Gradients have shapes too

The bookkeeping is the same, but now every quantity carries a shape — and the shapes are the easiest way to check the work.

$\partial\mathcal{L}/\partial T$ always has the **same shape** as T itself.

For a dense layer $\mathbf{h} = \mathbf{x}W$ with $\mathbf{x} \in \mathbb{R}^{N \times d}$ and $W \in \mathbb{R}^{d \times m}$:

$$\frac{\partial\mathcal{L}}{\partial W} = \underbrace{\mathbf{x}^\top}_{\langle d \times N \rangle} \underbrace{\frac{\partial\mathcal{L}}{\partial \mathbf{h}}}_{\langle N \times m \rangle} \quad \frac{\partial\mathcal{L}}{\partial \mathbf{x}} = \underbrace{\frac{\partial\mathcal{L}}{\partial \mathbf{h}}}_{\langle N \times m \rangle} \underbrace{W^\top}_{\langle m \times d \rangle}$$

$\langle d \times m \rangle \checkmark$ $\langle N \times d \rangle \checkmark$

Writing the first one entry by entry shows what the matrix product is doing. The sum runs over the N examples, and each term is a product of two factors:

$$\left[\frac{\partial\mathcal{L}}{\partial W} \right]_{ij} = \sum_{n=1}^N \underbrace{x_{in}^\top}_{\text{how changing } W_{ij} \text{ changes } h_j \text{ in example } n} \cdot \underbrace{\left[\frac{\partial\mathcal{L}}{\partial \mathbf{h}} \right]_{nj}}_{\text{how much of the loss } h_j \text{ is responsible for}}$$

So each weight's gradient is *activity* $\times$ *responsibility*, summed over the batch: W_{ij} is the single wire from feature i to unit j , it is used once per example, and the fan-out rule from §5 says those N contributions add.

This is where the transposes in code come from. For an elementwise activation $\mathbf{h} = \sigma(\mathbf{z})$, recall from Lecture 2 that the Jacobian is *diagonal* — so the matrix multiply collapses into a Hadamard (elementwise) product:

$$\frac{\partial\mathcal{L}}{\partial \mathbf{z}} = \frac{\partial\mathcal{L}}{\partial \mathbf{h}} \odot \sigma'(\mathbf{z}).$$

This also answers the question of where the batch went: $\mathbf{x}$ and $\mathbf{h}$ both carry N , but W does not, because the sum above contracts it away.

7. Dynamic graphs and modules

Note that this can be assembled **dynamically**: the graph is built *by* the forward pass, rather than declared in advance. Calling `.backward()` then walks that record in reverse.

In code, each block is a **module** — which is a class — and we can chain these together.

(See the notebook and the textbook.)